



UNIVERSIDAD CENTRAL DEL ECUADOR
FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS
CARRERA DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

MANUAL TÉCNICO

Desarrollo de un Sistema Web para la Evaluación y Clasificación Automática de Hojas de
Vida Implementando Inteligencia Artificial

Autoras:

Meybili Tatiana Olivares Intriago
Lesly Salomé Salas Cueva

Tutora: Ing. Ximena Teresa Guerrón Sandoval, PhD.

Quito - 2026

ÍNDICE

Índice de tablas	4
Índice de figuras	5
1. INTRODUCCIÓN.....	6
1.1. Propósito	6
1.2. Objetivos	6
1.3. Alcance.....	6
1.4. Público objetivo.....	7
2. DESCRIPCIÓN TÉCNICA DEL SISTEMA.....	7
2.1. Entradas, procesamiento y salidas	7
2.2. Restricciones técnicas del prototipo.....	8
2.3. Atributos de calidad considerados	8
3. ARQUITECTURA DEL SOFTWARE.....	8
3.1. Capa cliente	9
3.2. Capa servidor y API	9
3.3. Capa de procesamiento	9
3.4. Capa de modelado	9
3.5. Capa de persistencia	10
3.6. Capa de resultados	10
3.7. Componentes del frontend.....	10
3.8. Componentes del backend.....	11
3.9. Flujo técnico de extremo a extremo.....	12
3.10. Vista de despliegue.....	12
4. TECNOLOGÍAS Y DEPENDENCIAS	13
5. ESTRUCTURA DEL PROGRAMA FUENTE	14
5.1. Backend	14
5.2. Frontend.....	15
6. MÓDULOS Y COMPONENTES INTERNOS	15
6.1. Tratamiento documental.....	16
6.2. Separación entre operación y analítica	17
7. BASE DE DATOS Y ALMACENAMIENTO	17

7.1. Bloques del modelo de datos	18
7.2. Entidades principales	18
7.3. Relaciones y cardinalidades	19
7.4. Contenedores de almacenamiento.....	20
7.5. Diccionario de datos	21
8. MODELO DE APRENDIZAJE AUTOMÁTICO	21
8.1. Variables utilizadas	21
8.2. Representación TF-IDF	21
8.3. Modelos evaluados	22
8.4. Entrenamiento y validación	22
8.5. Métricas de evaluación	22
8.6. Modelo seleccionado	23
8.7. Cálculo del score y ranking	23
8.8. Artefactos y ciclo de vida	23
9. API E INTEGRACIONES	24
9.1. Grupos funcionales de servicios.....	24
9.2. Catálogo de endpoints	24
9.3. Convenciones técnicas.....	25
9.4. Integración con Supabase	25
10. CONFIGURACIÓN TÉCNICA	25
11. SEGURIDAD Y PROTECCIÓN DE DATOS.....	26
11.1. Tratamiento de identificadores.....	26
11.2. Gestión de credenciales	27
11.3. Controles implementados y endurecimiento requerido	27
12. REGISTROS, ERRORES Y TRAZABILIDAD	28
12.1. Información mínima de una ejecución.....	28
12.2. Clasificación de errores técnicos.....	28
13. MANTENIMIENTO TÉCNICO	29
13.1. Mantenimiento de dependencias.....	29
13.2. Mantenimiento del modelo.....	29
13.3. Lista de verificación para cambios.....	29

14. GLOSARIO TÉCNICO.....	30
15. ANEXOS	31

Índice de tablas

Tabla 1. Entradas, procesamiento y salidas del sistema.....	7
Tabla 2. Atributos de calidad considerados.....	8
Tabla 3. Tecnologías y dependencias principales	13
Tabla 4. Estructura principal del backend	14
Tabla 5. Estructura principal del frontend.	15
Tabla 6. Módulos y componentes internos.	16
Tabla 7. Bloques del modelo de datos	18
Tabla 8. Entidades principales del sistema	19
Tabla 9. Relaciones y cardinalidades del modelo de datos	19
Tabla 10. Contenedores de almacenamiento	20
Tabla 11. Variables utilizadas por el modelo.	21
Tabla 12. Modelos de clasificación evaluados	22
Tabla 13. Métricas de evaluación.....	22
Tabla 14. Grupos funcionales de servicios	24
Tabla 15. Parámetros de configuración técnica	25
Tabla 16. Tratamiento de datos e identificadores	26
Tabla 17. Controles de seguridad y endurecimiento requerido	27
Tabla 18. Clasificación de errores técnicos	28
Tabla 19. Lista de verificación para cambios	30
Tabla 20. Glosario técnico	30
Tabla 21. Matriz de trazabilidad entre módulos y datos.	31
Tabla 22. Referencias documentales del sistema.....	31
Tabla 23. Diccionario de datos de <i>auto_training_check</i>	32
Tabla 24. Diccionario de datos de <i>candidate</i>	33
Tabla 25. Diccionario de datos de <i>candidate_pii</i>	33
Tabla 26. Diccionario de datos de <i>candidate_profile</i>	34
Tabla 27. Diccionario de datos de <i>candidate_source</i>	35
Tabla 28. Diccionario de datos de <i>classification_result</i>	37
Tabla 29. Diccionario de datos de <i>dataset_sample</i>	38
Tabla 30. Diccionario de datos de <i>execution_report</i>	39
Tabla 31. Diccionario de datos de <i>job_profile</i>	40
Tabla 32. Diccionario de datos de <i>model_version</i>	41
Tabla 33. Diccionario de datos de <i>model_version_event</i>	43
Tabla 34. Diccionario de datos de <i>processing_run</i>	44

Tabla 35. Diccionario de datos de retraining_check.....	45
Tabla 36. Diccionario de datos de training_run.	45
Tabla 37. Servicios generales y gestión documental.	47
Tabla 38. Dataset entrenamiento validación y versiones.	48
Tabla 39. Códigos de respuesta de la API.....	50

Índice de figuras

Figura 1. Arquitectura general del sistema.	9
Figura 2. Componentes funcionales del frontend.....	10
Figura 3. Componentes funcionales del backend.....	11
Figura 4. Flujo técnico del sistema.....	12
Figura 5. Despliegue lógico del prototipo en entorno local.	13
Figura 6. Diagrama entidad-relación del sistema.....	18

1. INTRODUCCIÓN

El presente Manual técnico documenta la estructura interna del Sistema de Clasificación de Hojas de Vida desarrollado como apoyo a la preselección de candidatos. Su contenido se fundamenta en la arquitectura, el modelo de datos, los componentes funcionales y el proceso de aprendizaje automático descritos en la tesis del proyecto.

El documento explica cómo se relacionan la interfaz web, la API, el procesamiento de documentos, el modelado y la persistencia. También establece una referencia para comprender el tratamiento de los datos, la generación de resultados, la trazabilidad y las actividades de mantenimiento técnico.

1.1. Propósito

Proporcionar al personal técnico una descripción organizada y verificable de la construcción y el funcionamiento interno del sistema, de modo que pueda analizar sus componentes, localizar responsabilidades, mantener la solución y realizar ampliaciones sin perder la trazabilidad del proceso de clasificación.

1.2. Objetivos

- Describir la arquitectura y la comunicación entre el *frontend*, el *backend*, los módulos de procesamiento y Supabase.
- Documentar las responsabilidades de los módulos, las entidades del modelo de datos y las estructuras de almacenamiento.
- Explicar el flujo técnico de extracción, normalización, vectorización, entrenamiento, inferencia y generación del *ranking*.
- Registrar los parámetros técnicos que condicionan el procesamiento y la evaluación de los modelos.
- Establecer consideraciones de seguridad, trazabilidad y mantenimiento aplicables al prototipo.

1.3. Alcance

El manual abarca el prototipo web implementado con React y Flask, la integración con Supabase, el procesamiento de hojas de vida en PDF con texto digital seleccionable, la representación TF-IDF, los modelos KNN y árbol de clasificación, el cálculo del *score* de afinidad, la generación del *ranking* y el registro de las ejecuciones.

No incluye instrucciones para operar las pantallas, instalar las herramientas de desarrollo ni iniciar los servicios. Tampoco constituye una especificación de infraestructura productiva, debido a que la solución evaluada corresponde a un prototipo ejecutado en un entorno controlado.

1.4. Público objetivo

Está dirigido a desarrolladores, responsables de soporte, administradores técnicos, personal encargado de la base de datos y profesionales que deban revisar o ampliar los componentes de procesamiento y aprendizaje automático.

2. DESCRIPCIÓN TÉCNICA DEL SISTEMA

La solución adopta una arquitectura cliente-servidor. El *frontend* presenta las funciones de gestión y consulta; el *backend* expone servicios REST, aplica validaciones y coordina los módulos internos; Supabase proporciona la base de datos PostgreSQL y el almacenamiento de archivos; y el componente de aprendizaje automático procesa las representaciones textuales para clasificar y ordenar los candidatos.

2.1. Entradas, procesamiento y salidas

La correspondencia entre las entradas, las etapas de procesamiento y los resultados técnicos se presenta en la Tabla 1.

Tabla 1. Entradas, procesamiento y salidas del sistema.

Elemento	Descripción técnica
Entradas	Perfil del puesto, criterios de evaluación y hojas de vida en PDF estándar.
Procesamiento documental	Validación, extracción de texto, depuración, normalización y seudonimización.
Procesamiento analítico	Construcción del <i>dataset</i> , vectorización TF-IDF, entrenamiento o inferencia y cálculo del <i>score</i> .
Salidas	Clasificación apto/no apto, <i>score</i> de 0 a 100, posición en el <i>ranking</i> , métricas y reportes de ejecución.
Persistencia	Perfiles, candidatos, fuentes, muestras, versiones de modelo, ejecuciones, resultados, reportes y archivos.

2.2. Restricciones técnicas del prototipo

- Solo se procesan archivos PDF con texto digital seleccionable; no se incorpora OCR.
- El documento debe conservar una estructura de una sola columna y los encabezados definidos para el formato estándar.
- No deben incluirse tablas complejas, cuadros de texto no convencionales, imágenes con texto ni certificados anexos dentro del mismo PDF.
- La carga admite como máximo 50 hojas de vida por vacante y 10 MB por archivo.
- El resultado automatizado apoya la preselección y no sustituye la decisión humana.

2.3. Atributos de calidad considerados

Los atributos de calidad tomados como referencia para el prototipo se sintetizan en la Tabla 2.

Tabla 2. Atributos de calidad considerados.

Atributo	Aplicación en el sistema
Confiabilidad	Registro del estado de cada ejecución, candidatos procesados y posibles fallos.
Trazabilidad	Relación entre perfil, documento, candidato, versión del modelo, ejecución y resultado.
Mantenibilidad	Separación entre interfaz, API, procesamiento, modelado y persistencia.
Interoperabilidad	Intercambio mediante servicios REST y estructuras de datos definidas.
Reproducibilidad	Registro del algoritmo, parámetros, versión del <i>dataset</i> , métricas y estados aleatorios.
Interpretabilidad	Presentación de clase, <i>score</i> normalizado y posición dentro del <i>ranking</i> .
Integridad	Uso de identificadores internos para mantener la correspondencia entre entradas y resultados.

3. ARQUITECTURA DEL SOFTWARE

La arquitectura se organiza en capas para separar la interacción del usuario, la coordinación de servicios, la preparación de los documentos, la clasificación automática y el almacenamiento como se puede ver en la Figura 1. Esta separación reduce el acoplamiento y permite modificar un componente sin concentrar toda la lógica en un único módulo.

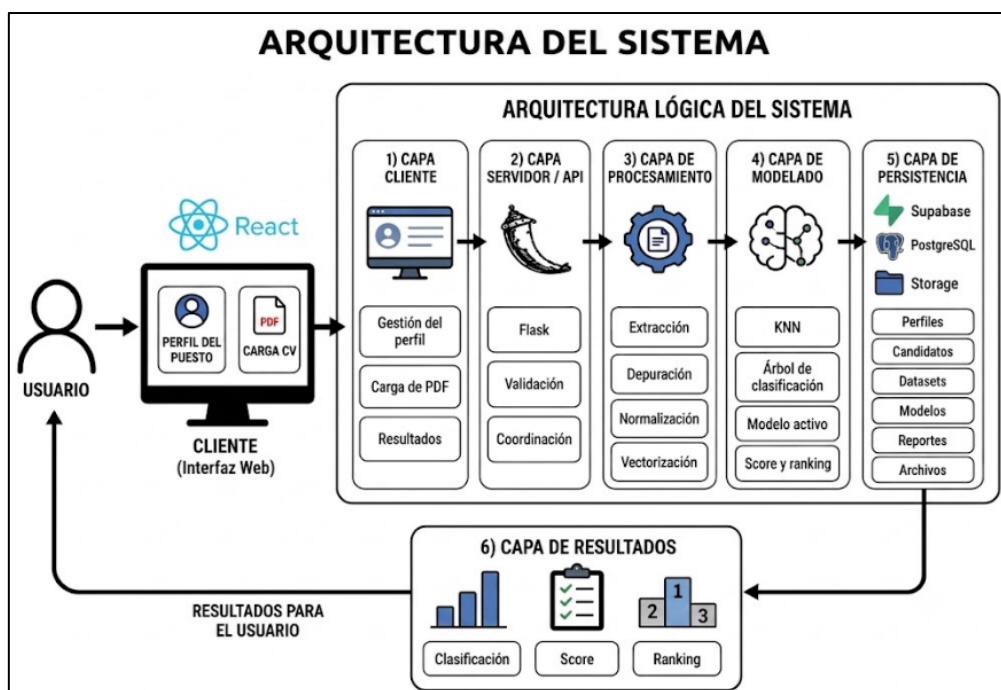


Figura 1. Arquitectura general del sistema.

3.1. Capa cliente

Implementada con React. Contiene la interfaz para gestionar perfiles, cargar documentos, consultar el procesamiento y visualizar resultados. No utiliza credenciales privadas ni se comunica directamente con la persistencia.

3.2. Capa servidor y API

Implementada con Python y Flask. Recibe solicitudes, valida entradas, aplica la lógica de negocio y coordina los servicios de procesamiento, modelado, persistencia y trazabilidad.

3.3. Capa de procesamiento

Extrae el texto seleccionable, elimina ruido, organiza la información por secciones, normaliza los campos y separa los identificadores personales antes del modelado.

3.4. Capa de modelado

Construye la representación TF-IDF, entrena y evalúa KNN y árbol de clasificación, recupera la versión activa y genera predicciones para nuevas hojas de vida.

3.5. Capa de persistencia

Utiliza Supabase para conservar datos estructurados en PostgreSQL y archivos en Storage. Mantiene las relaciones entre documentos, perfiles analíticos, modelos, ejecuciones y resultados.

3.6. Capa de resultados

Integra la etiqueta binaria, el *score* normalizado y la posición en el *ranking* que se presentan al usuario.

3.7. Componentes del frontend

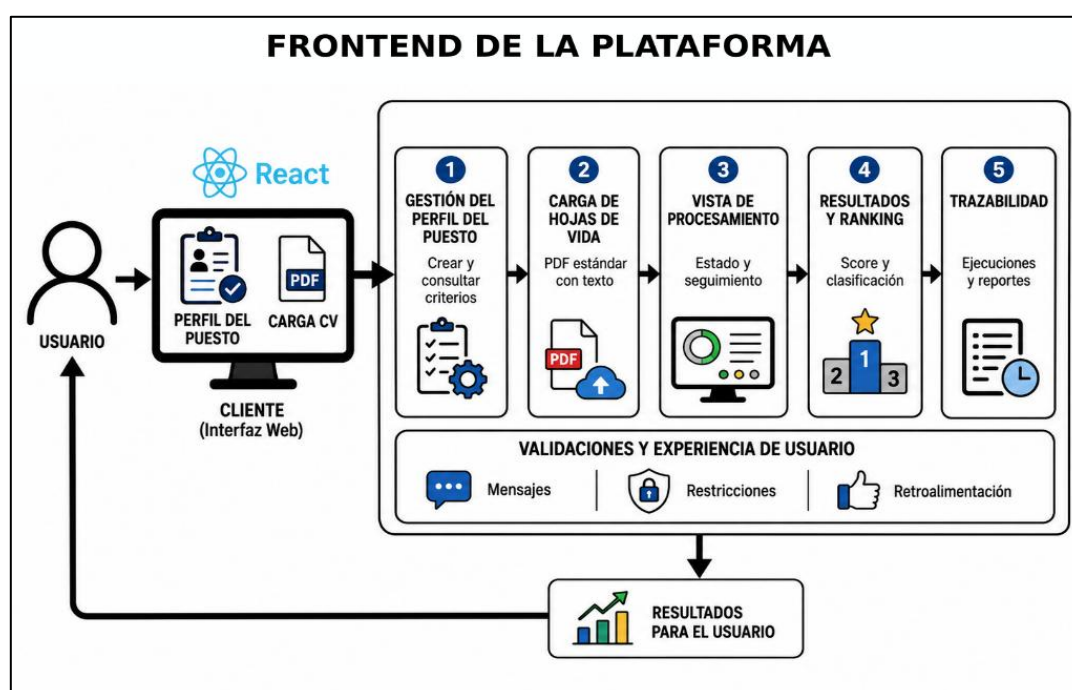


Figura 2. Componentes funcionales del frontend.

Los componentes visibles se concentran en cinco responsabilidades: gestión del perfil, carga de hojas de vida, seguimiento del procesamiento, presentación de resultados y consulta de trazabilidad. Las validaciones y los mensajes de retroalimentación atraviesan el flujo completo.

3.8. Componentes del backend

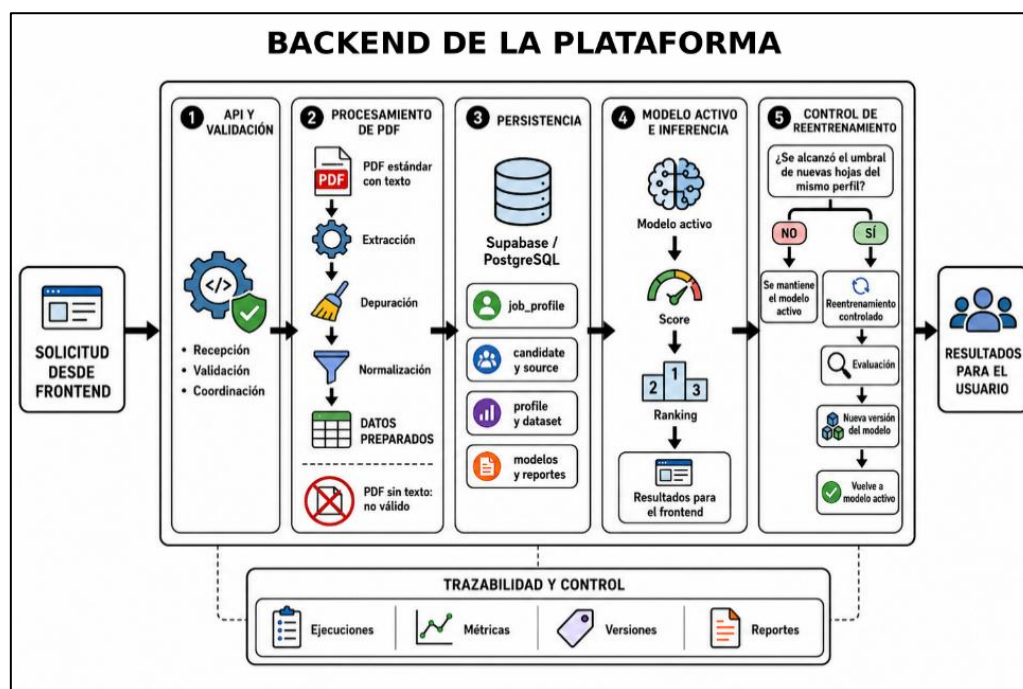


Figura 3. Componentes funcionales del backend.

El backend centraliza la recepción y validación de solicitudes, el procesamiento de PDF, la persistencia, la inferencia con el modelo activo, el control de reentrenamiento y el registro de trazabilidad. Esta organización evita que la interfaz contenga lógica de modelado o acceso privilegiado a los datos.

3.9. Flujo técnico de extremo a extremo

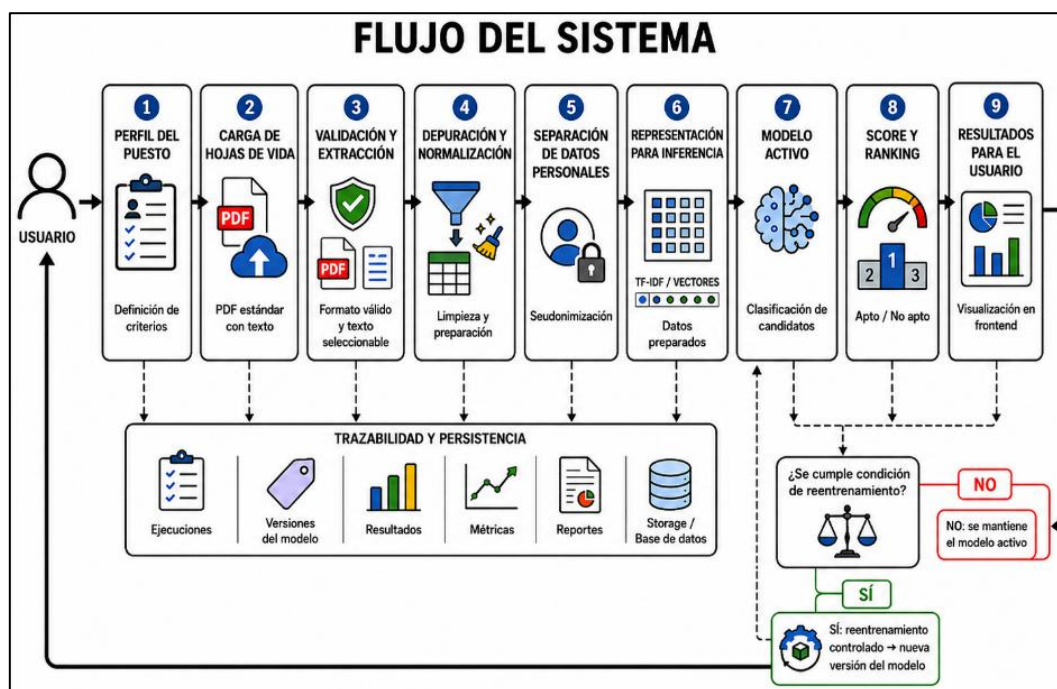


Figura 4. Flujo técnico del sistema.

- Definir el perfil del puesto y sus criterios.
- Recibir y validar las hojas de vida en PDF.
- Extraer, depurar y normalizar el contenido textual.
- Separar o enmascarar los datos personales que no intervienen en el modelo.
- Construir la representación TF-IDF requerida para la inferencia.
- Recuperar la versión activa y clasificar cada candidato.
- Calcular el *score*, ordenar los resultados y generar el *ranking*.
- Persistir la ejecución, versión, métricas, resultados y reportes.
- Verificar las condiciones establecidas para un reentrenamiento controlado.

3.10. Vista de despliegue

En el entorno evaluado, el navegador consume el *frontend* disponible localmente; este se comunica con la API Flask, que a su vez accede a Supabase mediante credenciales privadas configuradas únicamente en el servidor. Los procedimientos para iniciar estos componentes se encuentran en el Manual para la ejecución del programa fuente.

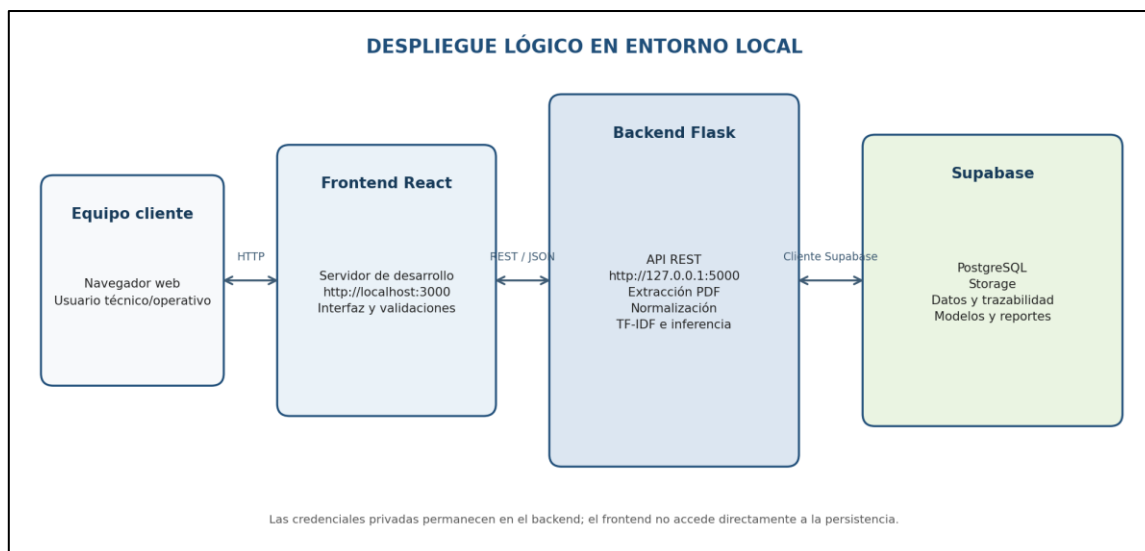


Figura 5. Despliegue lógico del prototipo en entorno local.

4. TECNOLOGÍAS Y DEPENDENCIAS

Las tecnologías se seleccionaron para integrar la interfaz, los servicios web, el procesamiento documental, el aprendizaje automático y la persistencia dentro de una solución modular. Las versiones instalables y su procedimiento de preparación se documentan en el Manual de instalación del software utilizado y en los archivos de dependencias del proyecto.

Las tecnologías se relacionan con su responsabilidad dentro de la solución en la Tabla 3.

Tabla 3. Tecnologías y dependencias principales

Componente	Tecnología	Responsabilidad
Frontend	React	Construcción de interfaces mediante componentes reutilizables.
Backend	Python	Lenguaje principal para servicios, procesamiento y modelado.
API	Flask	Exposición de servicios REST y coordinación de solicitudes.
Integración web	Flask-CORS	Control de solicitudes entre el frontend y el backend.
Configuración	python-dotenv	Carga de parámetros y credenciales desde variables de entorno.
Extracción PDF	pdfplumber	Obtención de texto digital seleccionable desde los documentos.
Datos	Pandas	Limpieza, consolidación y administración del dataset.

Cálculo numérico	NumPy	Operaciones con vectores y matrices.
Aprendizaje automático	Scikit-learn	TF-IDF, KNN, árbol de clasificación, partición, validación y métricas.
Artefactos	Joblib	Serialización y recuperación de modelos y vectorizadores.
Persistencia	Supabase / PostgreSQL	Datos estructurados, relaciones, auditoría y acceso a Storage.
Servidor WSGI	Gunicorn	Ejecución del backend en un esquema WSGI cuando corresponda.
Versionamiento	Git	Control de cambios y recuperación de versiones del código.

5. ESTRUCTURA DEL PROGRAMA FUENTE

El programa fuente se mantiene separado en un proyecto backend y un proyecto frontend. Esta división corresponde a la arquitectura cliente-servidor y permite administrar de forma independiente la interfaz, la API, el procesamiento y las dependencias de cada componente.

5.1. Backend

Los elementos principales del código fuente del backend y su responsabilidad se describen en la Tabla 4.

Tabla 4. Estructura principal del backend

Elemento	Responsabilidad
run.py	Punto de entrada que crea e inicia la aplicación Flask.
app/__init__.py	Fábrica de la aplicación, CORS, registro de blueprints y manejadores de errores.
app/config.py	Parámetros operativos, límites de carga y configuración del modelado.
app/api/	Rutas REST para carga, perfiles, dataset, entrenamiento, inferencia, versiones y reportes.
app/services/	Lógica de negocio, procesamiento documental, persistencia, modelado, ranking y reportes.
app/repositories/	Acceso a las tablas de PostgreSQL mediante el cliente de Supabase.
app/machine_learning/	Componentes comunes y entrenadores específicos de KNN y árbol de clasificación.
app/validators/	Validación de PDF, perfiles laborales y parámetros de entrenamiento.

app/errors/	Excepciones propias y respuestas HTTP normalizadas.
app/config/	Configuración JSON utilizada para la normalización del contenido.
app/utils/	Utilidades transversales, incluida la configuración de registros.
requirements.txt	Dependencias de Python requeridas por el backend.
.env.example	Plantilla de variables sin credenciales privadas.

5.2. Frontend

Los elementos principales del código fuente del *frontend* se detallan en la Tabla 5.

Tabla 5. Estructura principal del frontend.

Elemento	Responsabilidad
src/app/	Rutas y páginas principales construidas con Next.js.
src/app/job-profiles/	Pantalla para crear, consultar, editar y eliminar perfiles laborales.
src/app/results/	Presentación de resultados de clasificación y ranking.
src/app/execution-reports/	Consulta de trazabilidad y reportes de ejecución.
src/components/app-shell/	Estructura visual compartida de la aplicación.
src/components/processing/	Panel de seguimiento del estado de procesamiento.
src/components/providers/	Proveedores globales utilizados por la interfaz.
src/services/api/client.ts	Cliente HTTP común para comunicarse con el backend.
src/services/api/endpoints.ts	Definición centralizada de rutas de la API.
src/services/api/cv-upload/	Servicio de carga de hojas de vida.
src/services/api/job-profiles/	Servicios y hooks para perfiles laborales.
src/services/api/processing/	Servicios para consultar el procesamiento.
src/services/api/reports/	Servicios para obtener reportes y trazabilidad.
src/services/api/results/	Servicios para consultar resultados y ranking.
src/services/api/types/	Tipos TypeScript de solicitudes y respuestas.
src/services/react-query/	Configuración de consultas, caché y sincronización de datos.

6. MÓDULOS Y COMPONENTES INTERNOS

Los módulos se definen por la responsabilidad que cumplen dentro del flujo. Su comparación permite identificar dependencias y localizar las transformaciones realizadas durante el procesamiento.

Las entradas, transformaciones y salidas de los módulos internos se comparan en la Tabla 6.

Tabla 6. Módulos y componentes internos.

Módulo	Entrada	Procesamiento	Salida
Perfiles de puesto	Cargo, descripción y criterios	Valida y persiste la referencia de evaluación	Perfil disponible para dataset e inferencia
Carga de documentos	Archivos PDF	Valida formato, cantidad y tamaño	Fuentes documentales aceptadas o rechazadas
Extracción	PDF con texto seleccionable	Obtiene el contenido digital	Texto extraído
Normalización	Texto extraído	Depura ruido y organiza secciones	Perfil analítico normalizado
Seudonimización	Datos extraídos	Separa, elimina o enmascara identificadores	Registro analítico sin identificadores directos
Dataset	Perfil analítico y perfil de puesto	Consolida variables, etiqueta y partición	Muestras reproducibles y versionadas
Entrenamiento	Dataset y parámetros	Ajusta TF-IDF, KNN y árbol; calcula métricas	Versiónes y artefactos del modelo
Inferencia	Perfil, candidatos y modelo activo	Vectoriza y predice la clase	Clase y puntuación del modelo
Score y ranking	Predicción y similitud textual	Pondera, normaliza y ordena	Ranking Top 3, 5, 10 o 20
Trazabilidad	Eventos y resultados	Relaciona versiones, métricas y ejecución	Reportes y evidencia técnica
Reentrenamiento	Nuevos datos y verificaciones	Evalúa condiciones y registra un nuevo ciclo	Nueva versión candidata o permanencia del modelo activo

6.1. Tratamiento documental

La extracción depende de un formato estándar con las secciones: datos personales, perfil, formación académica, experiencia laboral, habilidades y tecnologías, idiomas, cursos y referencias. Después de la extracción se eliminan saltos innecesarios, fragmentos repetidos y ruido de maquetación; a continuación, la información se organiza en campos comparables.

El módulo debe rechazar o marcar como no procesable un PDF que carezca de texto seleccionable. Esta condición no representa un fallo del clasificador, sino una entrada fuera del alcance técnico definido para el prototipo.

6.2. Separación entre operación y analítica

La capa operativa conserva la relación con el documento y el candidato, mientras que la capa analítica contiene las características utilizadas por los modelos. Esta separación permite mantener trazabilidad sin introducir nombres, identificaciones, correos, teléfonos, direcciones o fotografías como variables de clasificación.

7. BASE DE DATOS Y ALMACENAMIENTO

El modelo de datos separa la gestión operativa, analítica, de modelos, de ejecución y de reentrenamiento. Los archivos se organizan en contenedores independientes. La relación entre estos bloques permite reconstruir el origen de una predicción y las condiciones bajo las cuales fue generada.

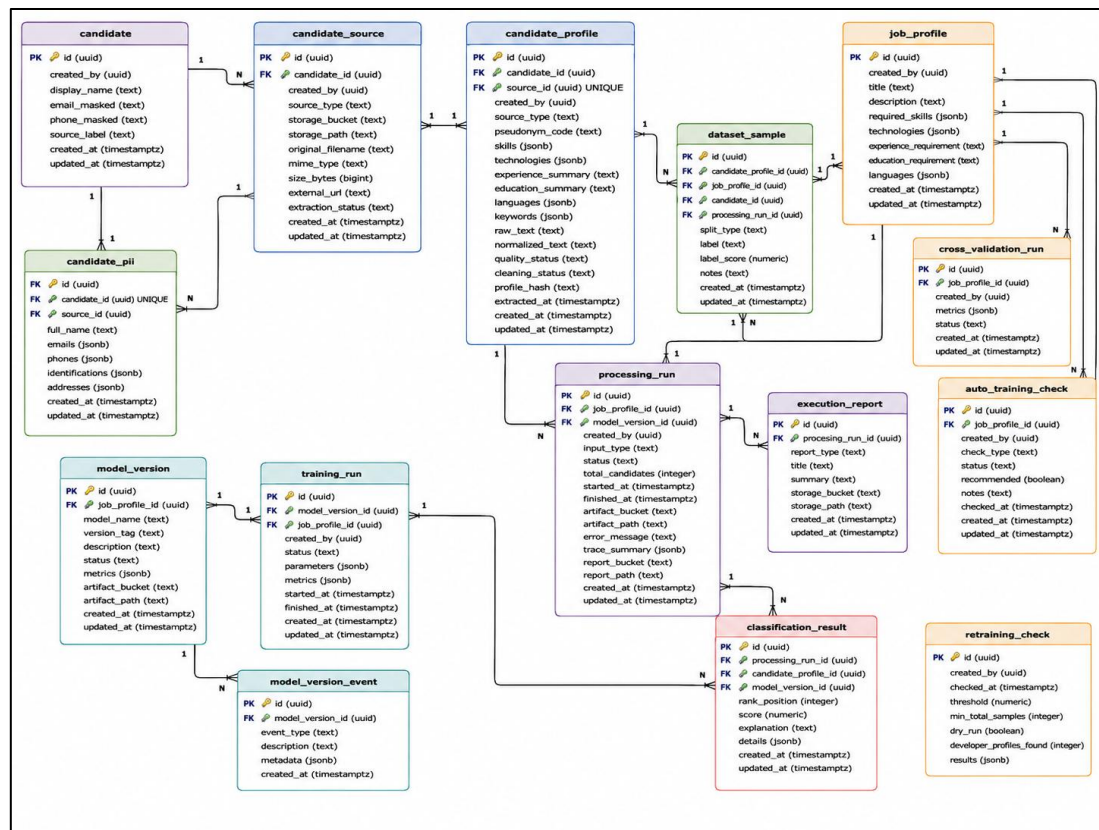


Figura 6. Diagrama entidad-relación del sistema.

7.1. Bloques del modelo de datos

La agrupación funcional de las entidades y estructuras de almacenamiento se muestra en la Tabla 7.

Tabla 7. Bloques del modelo de datos

Bloque	Entidades o estructuras
Gestión operativa	job_profile, candidate, candidate_source, candidate_pii
Gestión analítica	candidate_profile
Gestión de modelos	dataset_sample, model_version, training_run, model_version_event, cross_validation_run
Gestión de ejecución	processing_run, classification_result, execution_report
Gestión de reentrenamiento	retraining_check, auto_training_check
Almacenamiento	Buckets para PDF, textos normalizados, datasets, reportes y artefactos

7.2. Entidades principales

La función técnica de cada entidad principal se especifica en la Tabla 8.

Tabla 8. Entidades principales del sistema

Entidad	Función principal
job_profile	Perfil del puesto o vacante.
candidate	Registro operativo del candidato.
candidate_source	Fuente documental asociada al candidato.
candidate_profile	Perfil analítico normalizado.
candidate_pii	Datos personales separados.
dataset_sample	Muestra etiquetada para entrenamiento, prueba o validación.
model_version	Versión del modelo entrenado.
training_run	Entrenamiento, métricas y parámetros.
processing_run	Ejecución de procesamiento o inferencia.
classification_result	Resultado individual de clasificación.
model_version_event	Evento del ciclo de vida del modelo.
cross_validation_run	Resultado de validación cruzada.
retraining_check	Verificación general de reentrenamiento.
auto_training_check	Verificación automática por perfil.
execution_report	Reporte técnico o de resultados.

7.3. Relaciones y cardinalidades

Las relaciones y cardinalidades que permiten mantener la trazabilidad se presentan en la Tabla 9.

Tabla 9. Relaciones y cardinalidades del modelo de datos

Relación	Cardinalidad	Descripción
candidate -> candidate_source	1:N	Un candidato puede tener una o varias fuentes.
candidate -> candidate_profile	1:N	Un candidato puede tener varios perfiles analíticos.
candidate -> candidate_pii	1:1	Los datos personales se mantienen en un registro separado.
candidate_source -> candidate_profile	1:1	Una fuente puede generar un perfil analítico.
candidate_source -> candidate_pii	1:N	Una fuente puede asociarse con datos personales extraídos.
job_profile -> dataset_sample	1:N	Un perfil de puesto reúne múltiples muestras.

candidate_profile -> dataset_sample	1:N	Un perfil analítico puede evaluarse frente a distintos puestos.
processing_run -> dataset_sample	1:N	Una ejecución puede generar o asociar muestras.
model_version -> training_run	1:N	Una versión puede registrar entrenamientos.
job_profile -> training_run	1:N	Un perfil puede tener varias ejecuciones de entrenamiento.
job_profile -> model_version	1:N	Un perfil puede asociarse con varias versiones.
job_profile -> processing_run	1:N	Un perfil puede originar múltiples ejecuciones.
model_version -> processing_run	1:N	Una versión puede utilizarse en varias inferencias.
processing_run -> classification_result	1:N	Cada ejecución genera resultados individuales.
candidate_profile -> classification_result	1:N	Un perfil analítico puede aparecer en varios rankings.
model_version -> classification_result	1:N	Una versión genera múltiples resultados.
model_version -> model_version_event	1:N	Una versión mantiene eventos de ciclo de vida.
job_profile -> cross_validation_run	1:N	Un perfil puede registrar varias validaciones.
job_profile -> auto_training_check	1:N	Un perfil puede tener verificaciones automáticas.
processing_run -> execution_report	1:N	Una ejecución puede producir varios reportes.

7.4. Contenedores de almacenamiento

La distribución de documentos, conjuntos de datos, artefactos y reportes se resume en la Tabla 10.

Tabla 10. Contenedores de almacenamiento

Contenedor	Contenido
cv-raw	Hojas de vida originales en PDF.
cv-normalized	Textos o archivos JSON normalizados.
datasets	Archivos CSV o JSON utilizados para entrenamiento y validación.
model-artifacts	Modelos serializados, vectorizadores y artefactos.
reports	Reportes de ranking, métricas y ejecución.

7.5. Diccionario de datos

El esquema de persistencia está conformado por 15 tablas y 215 columnas utilizadas para conservar los perfiles laborales, candidatos, fuentes documentales, perfiles analíticos, conjuntos de datos, versiones de modelos, ejecuciones, resultados y reportes. Debido a su extensión, el detalle de los campos, tipos de datos y restricciones se presenta en el *Anexo 3*.

8. MODELO DE APRENDIZAJE AUTOMÁTICO

El componente analítico resuelve una clasificación binaria: clase 1 para candidatos aptos o relevantes y clase 0 para candidatos no aptos o no relevantes. La unidad de análisis es una hoja de vida procesada frente a un perfil de puesto determinado.

8.1. Variables utilizadas

Los campos empleados y excluidos del proceso de modelado se identifican en la Tabla 11.

Tabla 11. Variables utilizadas por el modelo.

Campo	Propósito	Uso en el modelo
normalized_text	Texto consolidado para vectorización.	Sí
skills	Habilidades identificadas.	Sí
technologies	Tecnologías declaradas.	Sí
experience_summary	Resumen de experiencia.	Sí
education_summary	Resumen de formación.	Sí
languages	Idiomas identificados.	Sí
keywords	Palabras clave relevantes.	Sí
label	Clase objetivo.	Sí
rubric	Criterios de etiquetado.	No
split_name	Partición train/test/validation.	No
dataset_version	Versión del conjunto de datos.	No
pseudonym_code	Identificador seudonimizado.	No

8.2. Representación TF-IDF

El texto normalizado se transforma en una matriz numérica mediante TF-IDF. La configuración del prototipo utiliza un máximo de 5 000 características y n-gramas de una y dos palabras. El vectorizador utilizado durante el entrenamiento debe conservarse junto con el modelo, porque la inferencia debe aplicar la misma representación y el mismo vocabulario.

8.3. Modelos evaluados

Las características y configuraciones base de los algoritmos evaluados se comparan en la Tabla 12.

Tabla 12. Modelos de clasificación evaluados

Modelo	Funcionamiento	Configuración base
KNN	Clasifica según la clase predominante o la cercanía de los vecinos más próximos en el espacio TF-IDF.	3 vecinos.
Árbol de clasificación	Construye reglas mediante particiones sucesivas de las características.	Profundidad máxima 10; mínimo 3 muestras por hoja.

8.4. Entrenamiento y validación

- Partición principal: 80 % para entrenamiento y 20 % para prueba.
- Partición estratificada para conservar una proporción semejante entre las clases.
- Validación cruzada estratificada de cinco *folds*.
- Evaluación final complementaria mediante 100 ejecuciones con distintos *random states*.
- Métricas principales: *recall* y *f1-score*; métricas complementarias: *accuracy*, precisión y tiempo de entrenamiento/predicción.

8.5. Métricas de evaluación

Las métricas utilizadas para valorar los modelos y su interpretación se presentan en la Tabla 13.

Tabla 13. Métricas de evaluación

Métrica	Expresión	Interpretación
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Proporción total de clasificaciones correctas.
Precisión	$TP / (TP + FP)$	Confiabilidad de los candidatos marcados como aptos.
Recall	$TP / (TP + FN)$	Capacidad para recuperar candidatos realmente aptos.
F1-score	$2 \times (\text{Precisión} \times \text{Recall}) / (\text{Precisión} + \text{Recall})$	Equilibrio entre precisión y recuperación.

8.6. Modelo seleccionado

KNN fue seleccionado como modelo final. En las 100 ejecuciones con partición 80/20 obtuvo un *recall* promedio de 0,7720 y un *f1-score* de 0,6677, frente a 0,5870 y 0,5800 del árbol de clasificación. La validación cruzada de cinco particiones confirmó la tendencia, con *recall* de 0,7980 y *f1-score* de 0,6797 para KNN.

8.7. Cálculo del score y ranking

El *score* de afinidad combina la salida del modelo de clasificación con la similitud del coseno obtenida entre las representaciones TF-IDF del perfil laboral y del candidato:

$$S = 0,70 \times S_{\text{modelo}} + 0,30 \times S_{\text{similitud}}$$

Donde S_{modelo} representa el resultado generado por el modelo y $S_{\text{similitud}}$ corresponde a la similitud del coseno entre el perfil y la hoja de vida.

Posteriormente, los valores obtenidos dentro de una misma ejecución se normalizan mediante el método *min-max*:

$$\text{Score} = 100 \times \frac{S - S_{\min}}{S_{\max} - S_{\min}}$$

El valor final se utiliza para ordenar a los candidatos de mayor a menor afinidad y construir los *rankings* Top 3, Top 5, Top 10 o Top 20. Este resultado representa una posición relativa dentro de la ejecución y no debe interpretarse como una probabilidad de contratación.

8.8. Artefactos y ciclo de vida

Cada versión debe mantener asociados el vectorizador, el modelo serializado, el algoritmo, los hiperparámetros, la versión del *dataset*, las métricas, la matriz de confusión y la información de validación. Joblib permite guardar y recuperar los artefactos utilizados durante la inferencia.

Antes de activar una nueva versión, deben compararse sus resultados con el modelo vigente. La generación de una versión nueva no implica automáticamente su activación; se requiere conservar la trazabilidad de la evaluación y evitar que una actualización disminuya el desempeño.

9. API E INTEGRACIONES

La interfaz se comunica con el backend mediante servicios REST. El backend constituye el único punto de acceso a la lógica de procesamiento y a las credenciales privadas de Supabase. Las solicitudes intercambian datos estructurados y, cuando corresponde, documentos PDF; las respuestas devuelven estados, identificadores, resultados o información de error.

9.1. Grupos funcionales de servicios

Las responsabilidades de los servicios y las entidades relacionadas se organizan en la Tabla 14.

Tabla 14. Grupos funcionales de servicios

Dominio	Responsabilidad	Entidades relacionadas
Perfiles de puesto	Crear, consultar y actualizar la referencia utilizada en la evaluación.	job_profile
Carga y extracción	Validar documentos, registrar fuentes y extraer contenido.	candidate, candidate_source, candidate_pii
Normalización	Construir el perfil analítico seudonimizado.	candidate_profile
Dataset	Consolidar, etiquetar, particionar y versionar muestras.	dataset_sample
Entrenamiento	Entrenar modelos, calcular métricas y registrar versiones.	training_run, model_version, cross_validation_run
Inferencia y ranking	Clasificar, calcular score y ordenar candidatos.	processing_run, classification_result
Reportes	Consultar trazas y generar archivos JSON o CSV.	execution_report
Reentrenamiento	Verificar condiciones e iniciar un ciclo controlado.	retraining_check, auto_training_check

9.2. Catálogo de endpoints

El backend expone 33 endpoints bajo la ruta base /api/v1. Estos servicios permiten gestionar documentos, perfiles laborales, conjuntos de datos, entrenamientos, versiones de modelos, procesos de inferencia, rankings y reportes. El detalle de los métodos HTTP, rutas, entradas, salidas y posibles errores se presenta en el .

9.3. Convenciones técnicas

- La URL base del backend se administra mediante la configuración del frontend.
- La API valida la presencia y el formato de los datos antes de ejecutar los servicios internos.
- Los identificadores internos mantienen la relación entre recursos y evitan depender de nombres personales.
- Los errores deben expresarse mediante un código, un mensaje comprensible y el estado HTTP correspondiente.
- Las respuestas de procesos extensos deben conservar un identificador de ejecución para consultar su avance y trazabilidad.

9.4. Integración con Supabase

El backend utiliza el cliente de Supabase para acceder a PostgreSQL y Storage. La integración administra registros estructurados, documentos, *datasets*, artefactos y reportes. Las credenciales con privilegios no deben enviarse al navegador ni incluirse en respuestas de la API.

10. CONFIGURACIÓN TÉCNICA

La configuración separa los valores operativos, los límites de carga, los parámetros de modelado y las ubicaciones de artefactos. Este apartado documenta el efecto de los parámetros, mientras que su creación y asignación se explica en el Manual para la ejecución del programa fuente.

Los parámetros que condicionan la ejecución, el procesamiento y el modelado se detallan en la Tabla 15.

Tabla 15. Parámetros de configuración técnica

Parámetro	Valor base	Efecto técnico
MAX_CV_FILE_SIZE_MB	10	Tamaño máximo permitido por hoja de vida.
MAX_BATCH_FILES	50	Cantidad máxima de documentos por lote.
MAX_REQUEST_SIZE_MB	30	Límite técnico de la solicitud recibida por el backend.
TRAIN_TEST_SIZE	0.20	Proporción reservada para prueba.
TRAINING_RANDOM_STATE	42	Estado aleatorio base para reproducibilidad.

MIN_TRAINING_ROWS	10	Cantidad mínima para habilitar el entrenamiento.
TFIDF_MAX_FEATURES	5000	Número máximo de características TF-IDF.
TFIDF_NGRAM_RANGE	1,2	Uso de unigramas y bigramas.
KNN_DEFAULT_NEIGHBORS	3	Vecinos considerados por KNN.
TREE_MAX_DEPTH	10	Profundidad máxima del árbol.
TREE_MIN_SAMPLES_LEAF	3	Muestras mínimas requeridas por hoja.
DATASET_MIN_TEXT_LENGTH	80	Longitud mínima del texto utilizado para una muestra.
DATASET_POSITIVE_RATIO	0.45	Proporción positiva utilizada por la estrategia automática.
DATASET_MATCH_THRESHOLD	0.55	Umbral de coincidencia de apoyo para la construcción automática.
CROSS_VALIDATION_FOLDS	5	Número de particiones de validación cruzada.
MODEL_ARTIFACTS_DIR	artifacts	Directorio local de artefactos.
MODEL_ARTIFACT_BUCKET	model-artifacts	Contenedor de artefactos del modelo.
REPORTS_BUCKET	reports	Contenedor de reportes.

11. SEGURIDAD Y PROTECCIÓN DE DATOS

Las hojas de vida contienen datos personales. Por ello, el diseño separa la información necesaria para la operación de la información utilizada por el modelo, conforme a los principios de finalidad, minimización y seguridad considerados en la investigación.

11.1. Tratamiento de identificadores

El tratamiento previsto para los datos personales y analíticos se resume en la Tabla 16.

Tabla 16. Tratamiento de datos e identificadores

Dato	Tratamiento
Nombre y apellidos	Separación de la capa analítica y sustitución por identificador interno.
Correo electrónico	Enmascaramiento.
Teléfono	Enmascaramiento.
Número de identificación	Enmascaramiento.
Dirección	Exclusión del dataset analítico.

Fotografía	Exclusión del dataset analítico.
Nombre del archivo	Conservación operativa para trazabilidad.
Habilidades, experiencia, formación, tecnologías e idiomas	Conservación por su relación con el puesto.

11.2. Gestión de credenciales

- Mantener las llaves privadas únicamente en el backend y fuera del control de versiones.
- No incorporar secretos reales en manuales, capturas, repositorios o respuestas de la API.
- Utilizar plantillas de variables sin valores sensibles.
- Rotar una credencial cuando exista sospecha o evidencia de exposición.
- Aplicar el principio de privilegio mínimo a las cuentas y políticas de acceso.

11.3. Controles implementados y endurecimiento requerido

La diferencia entre los controles documentados y las medidas necesarias para un entorno productivo se presenta en la Tabla 17.

Tabla 17. Controles de seguridad y endurecimiento requerido

Control	Estado documentado	Recomendación
Separación de datos personales	Documentada e integrada al flujo analítico.	Verificar periódicamente que ningún nuevo campo personal ingrese al modelo.
Validación de PDF	Formato, cantidad, tamaño y texto seleccionable.	Incorporar análisis antimalware si se expone públicamente.
Credenciales en backend	La interfaz no recibe llaves privadas.	Aplicar rotación, secretos administrados y auditoría en producción.
Trazabilidad	Versiones, métricas, ejecuciones y resultados relacionados.	Definir retención, acceso y revisión de eventos.
Autenticación de usuarios	La tesis no especifica un mecanismo completo.	Implementar autenticación y autorización antes de un despliegue abierto.
Transporte seguro	No forma parte del entorno local evaluado.	Utilizar HTTPS y certificados válidos en producción.

12. REGISTROS, ERRORES Y TRAZABILIDAD

La trazabilidad permite relacionar una salida con el perfil, los candidatos, la versión del modelo y la ejecución que la produjo. El backend registra mensajes según el nivel configurado y conserva evidencia estructurada en las entidades de entrenamiento, procesamiento, resultados y reportes.

12.1. Información mínima de una ejecución

- Identificador y tipo de ejecución.
- Perfil de puesto evaluado.
- Candidatos y fuentes procesadas.
- Estado de la ejecución y duración interna.
- Versión del modelo y del dataset.
- Parámetros y métricas aplicables.
- Resultados individuales y posición en el ranking.
- Archivos o reportes generados.
- Mensajes de error asociados a documentos no procesados.

12.2. Clasificación de errores técnicos

Las categorías de error y el tratamiento esperado por el equipo de soporte se describen en la Tabla 18.

Tabla 18. Clasificación de errores técnicos

Categoría	Ejemplos	Tratamiento esperado
Validación de entrada	PDF no válido, tamaño o cantidad excedida, datos obligatorios ausentes.	Rechazar antes del procesamiento y devolver un mensaje específico.
Extracción	Documento sin texto seleccionable o estructura incompatible.	Marcar el documento y conservar la causa sin detener los demás archivos del lote.
Persistencia	Fallo de conexión, permisos o escritura.	Registrar la operación, evitar resultados huérfanos y permitir diagnóstico.
Modelado	Artefacto ausente, incompatible o versión no activa.	Impedir la inferencia y reportar la versión requerida.
Entrenamiento	Dataset insuficiente, inválido o con	Cancelar de forma controlada y

	una sola clase.	conservar la validación fallida.
Integración	Respuesta inesperada entre frontend, API o Supabase.	Registrar contexto técnico sin exponer secretos.

13. MANTENIMIENTO TÉCNICO

El mantenimiento debe conservar la correspondencia entre código, configuración, base de datos, artefactos y documentación. Todo cambio que modifique entradas, entidades, parámetros o respuestas debe acompañarse de pruebas y actualización del manual correspondiente.

13.1. Mantenimiento de dependencias

- Revisar los archivos requirements.txt, package.json y yarn.lock antes de actualizar bibliotecas.
- Probar la extracción de PDF, la conexión con Supabase y la carga de artefactos después de cada actualización.
- Evitar actualizaciones masivas sin verificación de compatibilidad, especialmente en Scikit-learn, Pandas, NumPy, Joblib y el cliente de Supabase.
- Registrar la versión probada de Python y de las dependencias cuando se genere una nueva versión del sistema.

13.2. Mantenimiento del modelo

- Ampliar el dataset con fuentes autorizadas y conservar el balance entre las clases.
- Revisar la consistencia de la rúbrica y los errores de clasificación antes de incorporar nuevas muestras.
- Mantener particiones estratificadas, múltiples estados aleatorios y validación cruzada.
- Evaluar conjuntos externos que no hayan intervenido en entrenamiento o selección.
- Comparar cada versión candidata con el modelo activo antes de cambiar su estado.
- Conservar parámetros, dataset, métricas, matriz de confusión, artefactos y fecha de activación.

13.3. Lista de verificación para cambios

Las comprobaciones mínimas posteriores a una modificación técnica se organizan en la Tabla 19.

Tabla 19. Lista de verificación para cambios

Área	Comprobación
Código	El cambio respeta la separación de responsabilidades y cuenta con pruebas.
Datos	Las migraciones conservan relaciones, auditoría e identificadores internos.
Privacidad	No se incorporan datos personales como características del modelo.
Modelo	Se comparan métricas y se conserva la versión anterior.
API	Las respuestas y errores mantienen compatibilidad o se versionan.
Documentación	Se actualiza el manual que corresponde, sin duplicar procedimientos.
Respaldo	Existe una copia recuperable de datos, artefactos y configuración no secreta.

14. GLOSARIO TÉCNICO

Los conceptos utilizados de manera recurrente en el manual se definen en la Tabla 20.

Tabla 20. Glosario técnico

Término	Definición
API REST	Interfaz de servicios basada en recursos y solicitudes HTTP.
Artefacto del modelo	Archivo generado durante el entrenamiento, como el modelo o el vectorizador.
Backend	Componente que implementa servicios, lógica, procesamiento y acceso a datos.
Bucket	Contenedor de archivos dentro de Supabase Storage.
Candidate profile	Representación analítica normalizada de una hoja de vida.
Dataset	Conjunto estructurado de muestras utilizado para entrenamiento y evaluación.
Endpoint	Punto de acceso de una API asociado con una operación o recurso.
F1-score	Media armónica entre precisión y recall.
Frontend	Componente de interfaz ejecutado en el navegador.
Inferencia	Uso de un modelo entrenado para clasificar nuevos registros.
KNN	Algoritmo que clasifica según los vecinos más cercanos.
OCR	Reconocimiento óptico de caracteres; no está incorporado en el prototipo.
Pipeline	Secuencia de etapas para transformar datos y producir una salida.
Recall	Proporción de casos positivos reales recuperados por el modelo.
Score de afinidad	Valor relativo de 0 a 100 utilizado para ordenar candidatos.
Seudonimización	Tratamiento que sustituye identificadores directos por códigos internos.

TF-IDF	Representación numérica que pondera la relevancia de términos en documentos.
Trazabilidad	Capacidad para reconstruir el origen y las condiciones de un resultado.
Vectorizador	Componente que transforma texto en vectores compatibles con el modelo.

15. ANEXOS

Anexo 1. Matriz de trazabilidad entre módulos y datos.

La relación entre módulos, entidades y almacenamiento se consolida en la Tabla 21.

Tabla 21. Matriz de trazabilidad entre módulos y datos.

Módulo	Entidades	Almacenamiento
Perfil de puesto	job_profile	Perfiles y criterios de evaluación.
Carga y extracción	candidate, candidate_source, candidate_pii	cv-raw
Normalización	candidate_profile	cv-normalized
Dataset	dataset_sample	datasets
Entrenamiento	training_run, model_version, cross_validation_run	model-artifacts y reports
Inferencia	processing_run, classification_result	reports
Reentrenamiento	retraining_check, auto_training_check, model_version_event	model-artifacts
Reporte	execution_report	reports

Anexo 2. Referencias documentales del sistema

Los documentos que deben consultarse para mantener y verificar la solución se indican en la Tabla 22.

Tabla 22. Referencias documentales del sistema

Documento	Uso
Tesis del proyecto	Fundamento de arquitectura, datos, modelado, evaluación y resultados.
Manual de usuario	Consulta del comportamiento visible de las funciones.
Manual de instalación del software utilizado	Preparación de herramientas y requisitos instalables.
Manual para la ejecución del	Configuración local, dependencias e inicio de servicios.

programa fuente	
Repositorio backend	Fuente autoritativa para rutas, servicios, dependencias y lógica de procesamiento.
Repositorio frontend	Fuente autoritativa para componentes, validaciones y consumo de la API.

Anexo 3. Diccionario de datos.

A. Tabla auto_training_check

Los campos y restricciones de la tabla auto_training_check se detallan en la **Tabla 23**.

Tabla 23. Diccionario de datos de auto_training_check.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del registro.
job_profile_id	uuid	FK	Identificador del perfil laboral relacionado.
active_model_version_id	uuid	FK	Versión activa al momento de la verificación.
created_by	uuid	—	Identificador opcional del usuario o proceso creador.
threshold	integer	—	Umbral utilizado para decidir el reentrenamiento.
new_profiles_count	integer	—	Cantidad de perfiles nuevos desde la última activación.
total_valid_profiles	integer	—	Cantidad total de perfiles válidos disponibles.
should_train	boolean	—	Indica si se cumplen las condiciones para entrenar.
training_executed	boolean	—	Indica si se ejecutó el entrenamiento.
selected_model_version_id	uuid	FK	Versión elegida por el proceso automático.
status	text	—	Estado actual del registro o proceso.

message	text	—	Mensaje descriptivo del resultado de la operación.
metadata	jsonb	—	Información complementaria almacenada como JSON.
created_at	timestamp with time zone	—	Fecha y hora de creación del registro.

B. Tabla candidate

Los campos y restricciones de la tabla candidate se detallan en la **Tabla 24**.

Tabla 24. Diccionario de datos de candidate.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del registro.
created_by	uuid	—	Identificador opcional del usuario o proceso creador.
display_name	text	—	Nombre visible o seudonimizado del candidato.
email_masked	text	—	Correo electrónico enmascarado.
phone_masked	text	—	Número telefónico enmascarado.
source_label	text	—	Etiqueta que identifica el origen del candidato.
created_at	timestamp with time zone	—	Fecha y hora de creación del registro.
updated_at	timestamp with time zone	—	Fecha y hora de la última actualización.

C. Tabla candidate_pii

Los campos y restricciones de la tabla candidate_pii se detallan en la **Tabla 25**.

Tabla 25. Diccionario de datos de candidate_pii.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del registro.
candidate_id	uuid	FK, UQ	Identificador del candidato relacionado.
source_id	uuid	FK	Identificador de la fuente documental relacionada.
full_name	text	—	Nombre completo extraído del documento.
emails	jsonb	—	Correos electrónicos personales almacenados como JSON.
phones	jsonb	—	Números telefónicos personales almacenados como JSON.
identifications	jsonb	—	Números de identificación almacenados como JSON.
addresses	jsonb	—	Direcciones personales extraídas y almacenadas como JSON.
created_at	timestamp with time zone	—	Fecha y hora de creación del registro.
updated_at	timestamp with time zone	—	Fecha y hora de la última actualización.

D. Tabla candidate_profile

Los campos y restricciones de la tabla candidate_profile se detallan en la **Tabla 26**.

Tabla 26. Diccionario de datos de candidate_profile.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del registro.
candidate_id	uuid	FK	Identificador del candidato relacionado.
source_id	uuid	FK	Identificador de la fuente documental relacionada.
created_by	uuid	—	Identificador opcional del usuario o proceso creador.
source_type	text	—	Tipo de fuente o mecanismo de generación.

pseudonym_code	text	—	Código seudonimizado utilizado en la capa analítica.
skills	jsonb	—	Habilidades normalizadas del candidato.
technologies	jsonb	—	Tecnologías requeridas o identificadas, según la tabla.
experience_summary	text	—	Resumen normalizado de la experiencia del candidato.
education_summary	text	—	Resumen normalizado de la formación del candidato.
languages	jsonb	—	Idiomas requeridos o identificados, según la tabla.
keywords	jsonb	—	Palabras clave normalizadas del candidato.
raw_text	text	—	Texto extraído antes de su normalización.
normalized_text	text	—	Texto preparado para análisis y vectorización.
quality_status	text	—	Estado de calidad del perfil o de la muestra.
cleaning_status	text	—	Estado de limpieza y normalización del texto.
profile_hash	text	—	Huella utilizada para identificar cambios en el perfil.
extracted_at	timestamp with time zone	—	Fecha y hora de extracción del contenido.
created_at	timestamp with time zone	—	Fecha y hora de creación del registro.
updated_at	timestamp with time zone	—	Fecha y hora de la última actualización.

E. Tabla candidate_source

Los campos y restricciones de la tabla candidate_source se detallan en la **Tabla 27**.

Tabla 27. Diccionario de datos de candidate_source.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del registro.
candidate_id	uuid	FK	Identificador del candidato relacionado.
created_by	uuid	—	Identificador opcional del usuario o proceso creador.
source_type	text	—	Tipo de fuente o mecanismo de generación.
storage_bucket	text	—	Contenedor de Storage del archivo original.
storage_path	text	—	Ruta del archivo original en Storage.
original_filename	text	—	Nombre original del archivo cargado.
mime_type	text	—	Tipo MIME del archivo cargado.
size_bytes	bigint	—	Tamaño del archivo expresado en bytes.
external_url	text	—	URL externa asociada a la fuente documental.
normalized_bucket	text	—	Contenedor del artefacto textual normalizado.
normalized_path	text	—	Ruta del artefacto textual normalizado.
extracted_text_length	integer	—	Número de caracteres extraídos del documento.
extracted_at	timestamp with time zone	—	Fecha y hora de extracción del contenido.
extraction_error	text	—	Mensaje producido cuando falla la extracción.
extraction_status	text	—	Estado del proceso de extracción.
created_at	timestamp with time zone	—	Fecha y hora de creación del registro.
updated_at	timestamp with time	—	Fecha y hora de la última actualización.

	zone		
--	------	--	--

F. Tabla `classification_result`

Los campos y restricciones de la tabla `classification_result` se detallan en la **Tabla 28**.

Tabla 28. Diccionario de datos de `classification_result`.

Campo	Tipo	Clave	Descripción
<code>id</code>	<code>uuid</code>	PK	Identificador único del registro.
<code>processing_run_id</code>	<code>uuid</code>	FK	Identificador de la ejecución de procesamiento.
<code>candidate_profile_id</code>	<code>uuid</code>	FK	Identificador del perfil analítico del candidato.
<code>model_version_id</code>	<code>uuid</code>	FK	Identificador de la versión del modelo relacionada.
<code>created_by</code>	<code>uuid</code>	—	Identificador opcional del usuario o proceso creador.
<code>predicted_label</code>	<code>boolean</code>	—	Clase binaria predicha para el candidato.
<code>score_0_100</code>	<code>numeric</code>	—	Score normalizado del candidato en escala de 0 a 100.
<code>rank_position</code>	<code>integer</code>	—	Posición obtenida dentro del ranking.
<code>match_summary</code>	<code>jsonb</code>	—	Detalle estructurado de las coincidencias encontradas.
<code>created_at</code>	<code>timestamp with time zone</code>	—	Fecha y hora de creación del registro.
<code>updated_at</code>	<code>timestamp with time zone</code>	—	Fecha y hora de la última actualización.

G. Tabla `cross_validation_run`

Los campos y restricciones de la tabla `cross_validation_run` se detallan en la Tabla 11.

Tabla 11. Diccionario de datos de cross_validation_run

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del registro.
job_profile_id	uuid	FK	Identificador del perfil laboral relacionado.
created_by	uuid	—	Identificador opcional del usuario o proceso creador.
requested_folds	integer	—	Número de particiones solicitadas para validación cruzada.
total_samples	integer	—	Cantidad total de muestras utilizadas.
selected_algorithm	text	—	Algoritmo seleccionado después de la comparación.
results	jsonb	—	Resultados estructurados de la verificación o validación.
created_at	timestamp with time zone	—	Fecha y hora de creación del registro.

H. Tabla dataset_sample

Los campos y restricciones de la tabla dataset_sample se detallan en la **Tabla 29**.

Tabla 29. Diccionario de datos de dataset_sample.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del registro.
candidate_profile_id	uuid	FK	Identificador del perfil analítico del candidato.
job_profile_id	uuid	FK	Identificador del perfil laboral relacionado.
candidate_id	uuid	FK	Identificador del candidato relacionado.
source_id	uuid	FK	Identificador de la fuente documental relacionada.

processing_run_id	uuid	FK	Identificador de la ejecución de procesamiento.
created_by	uuid	—	Identificador opcional del usuario o proceso creador.
pseudonym_code	text	—	Código seudonimizado utilizado en la capa analítica.
label	boolean	—	Clase objetivo asignada a la muestra.
label_reason	text	—	Justificación de la etiqueta asignada.
rubric	jsonb	—	Detalle de los criterios utilizados para etiquetar la muestra.
notes	text	—	Observaciones técnicas adicionales.
labeled_by	uuid	—	Identificador del responsable del etiquetado.
split_name	text	—	Partición del dataset asignada a la muestra.
source_type	text	—	Tipo de fuente o mecanismo de generación.
match_score	numeric	—	Puntaje de coincidencia entre candidato y perfil.
label_strategy	text	—	Estrategia empleada para asignar la etiqueta.
quality_status	text	—	Estado de calidad del perfil o de la muestra.
dataset_version	text	—	Versión del conjunto de datos asociada.
auto_generated	boolean	—	Indica si la muestra fue construida automáticamente.
created_at	timestamp with time zone	—	Fecha y hora de creación del registro.
updated_at	timestamp with time zone	—	Fecha y hora de la última actualización.

I. Tabla execution_report

Los campos y restricciones de la tabla execution_report se detallan en la **Tabla 30**.

Tabla 30. Diccionario de datos de execution_report.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del registro.

processing_run_id	uuid	FK	Identificador de la ejecución de procesamiento.
job_profile_id	uuid	FK	Identificador del perfil laboral relacionado.
model_version_id	uuid	FK	Identificador de la versión del modelo relacionada.
created_by	uuid	—	Identificador opcional del usuario o proceso creador.
report_type	text	—	Tipo funcional del reporte generado.
report_format	text	—	Formato de salida del reporte, JSON o CSV.
total_candidates	integer	—	Cantidad total de candidatos de la ejecución.
successful_candidates	integer	—	Cantidad de candidatos procesados correctamente.
failed_candidates	integer	—	Cantidad de candidatos cuyo procesamiento falló.
artifact_bucket	text	—	Contenedor de Storage donde se guarda el artefacto.
artifact_path	text	—	Ruta del artefacto dentro del contenedor.
generated_at	timestamp with time zone	—	Fecha y hora de generación del reporte.
metadata	jsonb	—	Información complementaria almacenada como JSON.
created_at	timestamp with time zone	—	Fecha y hora de creación del registro.

J. Tabla job_profile

Los campos y restricciones de la tabla job_profile se detallan en la **Tabla 31**.

Tabla 31. Diccionario de datos de job_profile.

Campo	Tipo	Clave	Descripción
-------	------	-------	-------------

id	uuid	PK	Identificador único del registro.
created_by	uuid	—	Identificador opcional del usuario o proceso creador.
title	text	—	Título del perfil laboral.
description	text	—	Descripción del perfil laboral.
required_skills	jsonb	—	Habilidades obligatorias del perfil laboral.
technologies	jsonb	—	Tecnologías requeridas o identificadas, según la tabla.
experience_requirement	text	—	Requisito de experiencia del perfil laboral.
education_requirement	text	—	Requisito de formación del perfil laboral.
languages	jsonb	—	Idiomas requeridos o identificados, según la tabla.
created_at	timestamp with time zone	—	Fecha y hora de creación del registro.
updated_at	timestamp with time zone	—	Fecha y hora de la última actualización.

K. Tabla model_version

Los campos y restricciones de la tabla model_version se detallan en la **Tabla 32**.

Tabla 32. Diccionario de datos de model_version.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del registro.
created_by	uuid	—	Identificador opcional del usuario o proceso creador.
job_profile_id	uuid	FK	Identificador del perfil laboral relacionado.

model_name	text	—	Nombre descriptivo de la versión del modelo.
algorithm	text	—	Algoritmo de clasificación utilizado.
vectorizer	text	—	Técnica de vectorización asociada al modelo.
version_tag	text	UQ	Etiqueta única de identificación de la versión.
status	text	—	Estado actual del registro o proceso.
artifact_bucket	text	—	Contenedor de Storage donde se guarda el artefacto.
artifact_path	text	—	Ruta del artefacto dentro del contenedor.
metrics	jsonb	—	Métricas de evaluación almacenadas como JSON.
dataset_version	text	—	Versión del conjunto de datos asociada.
train_size	integer	—	Cantidad de muestras utilizadas para entrenamiento.
test_size	integer	—	Cantidad de muestras reservadas para prueba.
model_parameters	jsonb	—	Hiperparámetros del modelo almacenados como JSON.
dataset_summary	jsonb	—	Resumen estructurado del conjunto de datos utilizado.
selection_reason	text	—	Justificación de la selección del modelo.
activated_at	timestamp with time zone	—	Fecha y hora de activación de la versión.
archived_at	timestamp with time zone	—	Fecha y hora en que la versión fue archivada.
created_at	timestamp with time	—	Fecha y hora de creación del registro.

	zone		
updated_at	timestamp with time zone	—	Fecha y hora de la última actualización.

L. Tabla model_version_event

Los campos y restricciones de la tabla model_version_event se detallan en la **Tabla 33**.

Tabla 33. Diccionario de datos de model_version_event.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del registro.
model_version_id	uuid	FK	Identificador de la versión del modelo relacionada.
job_profile_id	uuid	FK	Identificador del perfil laboral relacionado.
created_by	uuid	—	Identificador opcional del usuario o proceso creador.
event_type	text	—	Tipo de evento registrado sobre la versión.
previous_status	text	—	Estado de la versión antes del evento.
new_status	text	—	Estado asignado después del evento.
message	text	—	Mensaje descriptivo del resultado de la operación.
metadata	jsonb	—	Información complementaria almacenada como JSON.
created_at	timestamp with time zone	—	Fecha y hora de creación del registro.

M. Tabla processing_run

Los campos y restricciones de la tabla processing_run se detallan en la **Tabla 34**.

Tabla 34. Diccionario de datos de *processing_run*.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del registro.
job_profile_id	uuid	FK	Identificador del perfil laboral relacionado.
model_version_id	uuid	FK	Identificador de la versión del modelo relacionada.
created_by	uuid	—	Identificador opcional del usuario o proceso creador.
input_type	text	—	Tipo de entrada utilizada por la ejecución.
status	text	—	Estado actual del registro o proceso.
total_candidates	integer	—	Cantidad total de candidatos de la ejecución.
started_at	timestamp with time zone	—	Fecha y hora de inicio de la ejecución.
finished_at	timestamp with time zone	—	Fecha y hora de finalización de la ejecución.
artifact_bucket	text	—	Contenedor de Storage donde se guarda el artefacto.
artifact_path	text	—	Ruta del artefacto dentro del contenedor.
error_message	text	—	Detalle del error ocurrido durante la ejecución.
trace_summary	jsonb	—	Resumen estructurado de trazabilidad.
report_bucket	text	—	Contenedor de Storage utilizado para el reporte.
report_path	text	—	Ruta del reporte dentro del contenedor.
created_at	timestamp with time	—	Fecha y hora de creación del registro.

	zone		
updated_at	timestamp with time zone	—	Fecha y hora de la última actualización.

N. Tabla retraining_check

Los campos y restricciones de la tabla retraining_check se detallan en la **Tabla 35**.

Tabla 35. Diccionario de datos de retraining_check.

Campo	Tipo	Clave	Descripción
id	uuid	PK	Identificador único del registro.
created_by	uuid	—	Identificador opcional del usuario o proceso creador.
checked_at	timestamp with time zone	—	Fecha y hora de la verificación de reentrenamiento.
threshold	integer	—	Umbral utilizado para decidir el reentrenamiento.
min_total_samples	integer	—	Cantidad mínima total requerida para reentrenar.
dry_run	boolean	—	Indica si la verificación se ejecutó sin aplicar cambios.
developer_profiles_found	integer	—	Cantidad de perfiles válidos detectados.
results	jsonb	—	Resultados estructurados de la verificación o validación.

O. Tabla training_run

Los campos y restricciones de la tabla training_run se detallan en la **Tabla 36**.

Tabla 36. Diccionario de datos de training_run.

Campo	Tipo	Clave	Descripción
-------	------	-------	-------------

id	uuid	PK	Identificador único del registro.
model_version_id	uuid	FK	Identificador de la versión del modelo relacionada.
job_profile_id	uuid	FK	Identificador del perfil laboral relacionado.
created_by	uuid	—	Identificador opcional del usuario o proceso creador.
dataset_version	text	—	Versión del conjunto de datos asociada.
train_size	integer	—	Cantidad de muestras utilizadas para entrenamiento.
test_size	integer	—	Cantidad de muestras reservadas para prueba.
recall	numeric	—	Proporción de casos positivos reales recuperados.
f1_score	numeric	—	Media armónica entre precisión y recall.
precision	numeric	—	Proporción de positivos predichos correctamente.
accuracy	numeric	—	Proporción de predicciones correctas.
duration_ms	integer	—	Duración de la operación en milisegundos.
confusion_matrix	jsonb	—	Matriz de confusión obtenida en la evaluación.
selected_algorithm	text	—	Algoritmo seleccionado después de la comparación.
status	text	—	Estado actual del registro o proceso.
notes	text	—	Observaciones técnicas adicionales.
model_parameters	jsonb	—	Hiperparámetros del modelo almacenados como JSON.
dataset_summary	jsonb	—	Resumen estructurado del conjunto de datos utilizado.
metrics	jsonb	—	Métricas de evaluación almacenadas como JSON.
is_selected	boolean	—	Indica si el resultado fue seleccionado como

			modelo final.
created_at	timestamp with time zone	—	Fecha y hora de creación del registro.
updated_at	timestamp with time zone	—	Fecha y hora de la última actualización.

Anexo 4. Catálogo de endpoints de la API.

Los servicios correspondientes a servicios generales y gestión documental se detallan en la **Tabla 37**.

Tabla 37. Servicios generales y gestión documental.

Método	Ruta	Entrada	Salida	Errores
GET	/api/v1/health	No requiere entrada.	200. Estado y nombre del servicio.	500
POST	/api/v1/candidates/upload	multipart/form-data: file.	201. Candidato, fuente y ubicación del archivo.	400, 413, 500
POST	/api/v1/candidates/upload-batch	multipart/form-data: files.	201. Resumen del lote y resultado individual de cada archivo.	400, 413, 500
POST	/api/v1/candidate-sources/{source_id}/extract-text	Parámetro de ruta source_id.	200. Estado de extracción, fuente y datos del artefacto normalizado.	400, 404, 500
POST	/api/v1/candidate-sources/{source_id}/normalize-profile	Parámetro de ruta source_id.	200. Perfil analítico generado y secciones identificadas.	400, 404, 500
GET	/api/v1/candidate-sources/{source_id}/profile	Parámetro de ruta source_id.	200. Perfil normalizado asociado a la fuente.	404, 500
GET	/api/v1/candidate-sources/{source_id}/file	source_id; query opcional download.	200. Archivo PDF con tipo application/pdf.	400, 404, 500
POST	/api/v1/job-profiles	JSON del perfil laboral.	201. Perfil laboral creado.	400, 500
GET	/api/v1/job-profiles	Query opcional limit; valor predeterminado 10.	200. Lista de perfiles laborales.	400, 500

GET	/api/v1/job-profiles/{item_id}	Parámetro de ruta item_id.	200. Detalle del perfil laboral.	400, 500
PUT	/api/v1/job-profiles/{item_id}	item_id y JSON con campos modificables.	200. Perfil laboral actualizado.	400, 500
DELETE	/api/v1/job-profiles/{item_id}	Parámetro de ruta item_id.	200. Registro eliminado.	400, 500

Los servicios correspondientes a dataset entrenamiento validación y versiones se detallan en la **Tabla 38**.

Tabla 38. Dataset entrenamiento validación y versiones.

Método	Ruta	Entrada	Salida	Errores
POST	/api/v1/dataset-samples/build	JSON con job_profile_id y parámetros de construcción.	201. Resumen de muestras creadas, actualizadas y omitidas.	400, 404, 500
POST	/api/v1/dataset-samples/ensure-for-training	JSON con job_profile_id y dataset_version opcional.	200. Estado de disponibilidad o construcción del dataset.	400, 404, 500
POST	/api/v1/model-training/job-profiles/{job_profile_id}/train	JSON con versión del dataset y opciones de persistencia.	200. Métricas, modelos evaluados y versión activa.	400, 404, 500
POST	/api/v1/model-training/job-profiles/{job_profile_id}/train-iterations	JSON con iterations, base_random_state y opciones de persistencia.	200. Ejecuciones completadas, fallidas y reporte agregado.	400, 404, 500
GET	/api/v1/model-training/job-profiles/{job_profile_id}/all-executions	Query opcional limit.	200. Resumen estadístico de las ejecuciones de entrenamiento.	400, 500
POST	/api/v1/model-training/job-profiles/{job_profile_id}/all-executions/report	JSON con persist_to_storage y limit opcionales.	200. Reporte consolidado y ubicación del artefacto.	400, 500
POST	/api/v1/model-validation/job-profiles/{job_profile_id}/cross-validation	JSON con folds opcional.	200. Resultados de validación cruzada por algoritmo.	400, 404, 500
GET	/api/v1/model-versions/job-profiles/{job_profile_id}	Parámetro de ruta job_profile_id.	200. Versiones asociadas al perfil.	500

GET	/api/v1/model-versions/job-profiles/{job_profile_id}/active	Parámetro de ruta job_profile_id.	200. Versión activa del modelo.	404, 500
POST	/api/v1/model-versions/{model_version_id}/activate	JSON opcional con created_by y reason.	200. Versión activada y modelos anteriores archivados.	400, 404, 500
GET	/api/v1/model-versions/{model_version_id}/audit	Parámetro de ruta model_version_id.	200. Eventos de auditoría de la versión.	404, 500

Los servicios correspondientes a inferencia autoentrenamiento y reportes se detallan en la Tabla 3.

Tabla 3. Inferencia autoentrenamiento y reportes

Método	Ruta	Entrada	Salida	Errores
POST	/api/v1/inference/job-profiles/{job_profile_id}/candidate-profiles/{candidate_profile_id}	IDs de ruta; JSON con auto_retrain, persist_result y top_k.	200. Clasificación, score y datos de ejecución.	400, 404, 500
POST	/api/v1/inference/job-profiles/{job_profile_id}/batch	JSON con candidate_profile_ids y opciones de inferencia.	200. Clasificaciones y ranking del lote.	400, 404, 500
POST	/api/v1/inference/job-profiles/{job_profile_id}/rank-all	JSON con max_candidates, top_k y opciones de reutilización.	200. Ranking general del perfil laboral.	400, 404, 500
GET	/api/v1/inference/processing-runs/{processing_run_id}/ranking	Query opcional limit o top_k.	200. Ranking de la ejecución.	400, 404, 500
GET	/api/v1/inference/processing-runs/{processing_run_id}/results	Query opcional limit o top_k.	200. Resultados enriquecidos de los candidatos.	400, 404, 500
POST	/api/v1/auto-training/job-profiles/{job_profile_id}/check-and-run	JSON con enabled, force y created_by.	200. Verificación, decisión y resultado del autoentrenamiento.	400, 404, 500
GET	/api/v1/reports/processing-runs/{processing_run_id}/trace	Parámetro de ruta processing_run_id.	200. Trazabilidad completa de la ejecución.	404, 500
POST	/api/v1/reports/processing-runs/{processing_run_id}/generate	JSON con report_format y persist_to_storage.	200. Registro del reporte y evidencia técnica.	400, 404, 500

GET	/api/v1/reports/processing-runs/{processing_run_id}	Parámetro de ruta processing_run_id.	200. Reportes asociados a la ejecución.	500
GET	/api/v1/reports/{report_id}	Parámetro de ruta report_id.	200. Registro del reporte solicitado.	404, 500

Los códigos administrados por los manejadores globales se resumen en la **Tabla 39**.

Tabla 39. Códigos de respuesta de la API.

Código	Interpretación
200	Operación completada correctamente.
201	Recurso creado o carga procesada correctamente.
400	Solicitud, archivo o parámetro inválido.
404	Recurso solicitado no encontrado.
405	Método HTTP no permitido para la ruta.
413	La solicitud supera el límite MAX_REQUEST_SIZE_MB.
500	Excepción inesperada o fallo de integración.